experiment.py

```python
import numpy as np
import json
from sklearn.linear_model import LogisticRegression

RNG_SEED = 0


def simulate(n, rng):
    d = rng.uniform(0, 1, n)
    t = rng.binomial(1, 0.5, n)
    p_correct = 0.9 - 0.6 * d
    ehat_correct = rng.binomial(1, p_correct, n).astype(bool)
    ehat = np.where(ehat_correct, t, 1 - t)
    u_correct = rng.binomial(1, 0.5, n).astype(bool)
    u = np.where(u_correct, t, 1 - t)
    c = rng.uniform(0, 1, n)
    return d, t, ehat, u, c


def features(d, c, u, ehat):
    agree = (ehat == u).astype(float)
    return np.column_stack([d, c, u.astype(float), ehat.astype(float), agree])


def reward(action_is_defer, u, ehat, t, c, mode, interp="linear", reward_noise=0.0, rng=None, step_thr=0.5,
           sigmoid_k=10.0):
    b = np.where(action_is_defer, u, ehat)
    r_agree = (b == u).astype(float)
    r_correct = (b == t).astype(float)
    if mode == "oracle":
        r = r_correct
    elif mode == "flat":
        r = r_agree
    elif mode == "confmod":
        if interp == "linear":
            w = c
        elif interp == "quadratic":
            w = c ** 2
        elif interp == "sqrt":
            w = np.sqrt(c)
        elif interp == "step":
            w = (c > step_thr).astype(float)
        elif interp == "sigmoid":
            w = 1.0 / (1.0 + np.exp(-sigmoid_k * (c - 0.5)))
        else:
            raise ValueError(interp)
        r = w * r_agree + (1 - w) * r_correct
    else:
        raise ValueError(mode)
    if reward_noise > 0 and rng is not None:
        r = r + rng.normal(0, reward_noise, size=r.shape)
    return r


def fit_and_eval(mode, seed, n_train=40000, n_test=20000, interp="linear", reward_noise=0.0, step_thr=0.5,
                 sigmoid_k=10.0):
    rng = np.random.default_rng(seed)
    d, t, ehat, u, c = simulate(n_train, rng)
    r_defer = reward(np.ones(n_train, dtype=bool), u, ehat, t, c, mode, interp, reward_noise, rng, step_thr, sigmoid_k)
    r_stay = reward(np.zeros(n_train, dtype=bool), u, ehat, t, c, mode, interp, reward_noise, rng, step_thr, sigmoid_k)
    label = (r_defer > r_stay).astype(int)

    X = features(d, c, u, ehat)
    if label.min() == label.max():
        const_action = label[0]
        clf = None
    else:
        clf = LogisticRegression()
```

```
        clf.fit(X, label)
        const_action = None

    d2, t2, ehat2, u2, c2 = simulate(n_test, np.random.default_rng(seed + 10_000_000))
    X2 = features(d2, c2, u2, ehat2)
    if clf is None:
        defer = np.full(n_test, const_action, dtype=bool)
    else:
        defer = clf.predict(X2).astype(bool)
    b = np.where(defer, u2, ehat2)

    acc = float(np.mean(b == t2))
    disagree = ehat2 != u2
    sycophancy = float(np.mean(defer[disagree])) if disagree.any() else float("nan")
    progressive = float(np.mean(defer[disagree] & (u2[disagree] == t2[disagree])))
    regressive = float(np.mean(defer[disagree] & (u2[disagree] != t2[disagree])))

    quartile_edges = np.quantile(c2[disagree], [0.25, 0.5, 0.75])
    qidx = np.digitize(c2[disagree], quartile_edges)
    q_syc = [float(np.mean(defer[disagree][qidx == i])) for i in range(4)]

    coef_c = float(clf.coef_[0][1]) if clf is not None else None
    return {
        "mode": mode, "seed": seed, "interp": interp, "reward_noise": reward_noise,
        "accuracy": acc, "sycophancy_rate": sycophancy,
        "progressive": progressive, "regressive": regressive,
        "quartile_sycophancy": q_syc, "coef_confidence": coef_c,
    }


if __name__ == "__main__":
    results = {}

    # --- Main run (seed=0, linear interpolation) matching paper's headline table ---
    main = {m: fit_and_eval(m, RNG_SEED) for m in ["oracle", "flat", "confmod"]}
    results["main_seed0"] = main
    for m, r in main.items():
        print(m, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])

    # --- Multi-seed robustness (10 seeds) ---
    seeds = list(range(10))
    multiseed = {m: [fit_and_eval(m, s) for s in seeds] for m in ["oracle", "flat", "confmod"]}
    results["multiseed"] = multiseed
    print("\n--- multi-seed summary (n=10 seeds) ---")
    for m, runs in multiseed.items():
        accs = np.array([r["accuracy"] for r in runs])
        sycs = np.array([r["sycophancy_rate"] for r in runs])
        q = np.array([r["quartile_sycophancy"] for r in runs])
        print(m, "acc=%.3f+/-%.3f" % (accs.mean(), accs.std()),
              "syc=%.3f+/-%.3f" % (sycs.mean(), sycs.std()),
              "q_mean=", ["%.3f" % x for x in q.mean(axis=0)],
              "q_std=", ["%.3f" % x for x in q.std(axis=0)])

    # --- Alternative functional forms of the confidence-modulation weight ---
    print("\n--- alternative interpolation functional forms (seed=0) ---")
    interp_results = {}
    for interp in ["linear", "quadratic", "sqrt", "step"]:
        r = fit_and_eval("confmod", RNG_SEED, interp=interp)
        interp_results[interp] = r
        print(interp, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["interp_forms"] = interp_results

    # --- Noisy reward robustness (breaks the "tautological by construction" concern:
    #     policy must generalize from noisy reward-optimal labels, not exact reward) ---
    print("\n--- reward-noise robustness (seed=0, linear interp) ---")
    noise_results = {}
    for noise in [0.0, 0.1, 0.25, 0.5]:
        r = fit_and_eval("confmod", RNG_SEED, reward_noise=noise)
```

```
        noise_results[noise] = r
        print("noise=%.2f" % noise, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["noise_robustness"] = noise_results

    # --- Diagnostic: is the approval-flat "policy" actually constant, as Section 2 of v2
    #     claimed, or does a non-trivial classifier get fit? (Reviewer-flagged discrepancy.) ---
    print("\n--- diagnostic: approval-flat imitation label is not constant ---")
    rng_diag = np.random.default_rng(RNG_SEED)
    d, t, ehat, u, c = simulate(40000, rng_diag)
    r_defer = reward(np.ones(40000, dtype=bool), u, ehat, t, c, "flat")
    r_stay = reward(np.zeros(40000, dtype=bool), u, ehat, t, c, "flat")
    flat_label = (r_defer > r_stay).astype(int)
    print("flat label min/max/mean:", flat_label.min(), flat_label.max(), float(flat_label.mean()))
    results["flat_label_diagnostic"] = {
        "min": int(flat_label.min()), "max": int(flat_label.max()), "mean": float(flat_label.mean())
    }

    # --- Threshold-shift check: does the "step" functional form's agreement with "linear"
    #     (Sec 3.3) come from shape, or just from both crossing w=0.5 at the same place?
    #     Shift the step threshold and see if quartile sycophancy tracks the crossing point. ---
    print("\n--- step-threshold shift (seed=0): does crossing point, not shape, drive Sec 3.3? ---")
    thr_results = {}
    for thr in [0.3, 0.5, 0.7]:
        r = fit_and_eval("confmod", RNG_SEED, interp="step", step_thr=thr)
        thr_results[thr] = r
        print("thr=%.1f" % thr, "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["step_threshold_shift"] = thr_results

    # --- Reviewer question 3: does *curvature* matter independent of crossing point?
    #     Sigmoid centered at c=0.5 (crossing fixed) with varying steepness k. If quartile
    #     sycophancy is ~invariant to k, the sweep confirms only crossing point ever mattered;
    #     if it varies with k, curvature has an independent, measurable effect. ---
    print("\n--- sigmoid steepness sweep (seed=0, crossing fixed at c=0.5): isolates shape from crossing point ---")
    sigmoid_results = {}
    for k in [1.0, 5.0, 10.0, 30.0]:
        r = fit_and_eval("confmod", RNG_SEED, interp="sigmoid", sigmoid_k=k)
        sigmoid_results[k] = r
        print("k=%.1f" % k, "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["sigmoid_steepness_shift"] = sigmoid_results

    # --- Print progressive/regressive breakdown directly from main_seed0 so numbers reported
    #     in the paper are read off this run, not hand-copied from an earlier version. ---
    print("\n--- progressive/regressive breakdown (seed=0, from main run) ---")
    for m, r in main.items():
        print(m, "progressive=%.4f" % r["progressive"], "regressive=%.4f" % r["regressive"])

    with open("results.json", "w") as f:
        json.dump(results, f, indent=2)
    print("\nWrote results.json")
```